OOP
ASSIGNMENT

NAME: AHMED SAEED
CLASS: BSCS-B
ROLL NO. 07
SEAT NO. EB26210006013
GITHUB:
https://github.com/Amdm130/CS-2nd-Semester-Assignments

Q. What is x86 with 64/32 bit?

x86 is the family name for the instruction set architecture that started with Intel's 8086 processor back in 1978 (the name comes from those early chips: 8086, 80286, 80386, 80486). Over time it's grown to support different "bitness". how much data the CPU can handle in one register/operation and how much memory it can address.

32-bit (x86 / IA-32)

- Uses 32-bit registers (like `EAX`, `EBX`)
- Can address a maximum of 4 GB of RAM (2^{32} bytes)
- Introduced with the 80386 in 1985
- Often just called "x86" when contrasted with the 64-bit version

64-bit (x86-64 / x64 / AMD64 / Intel 64)

- Uses 64-bit registers (like `RAX`, `RBX`)
- Can theoretically address up to 16 exabytes of memory (in practice, OS/hardware limits bring it down to terabytes)
- Introduced by AMD in 2003 (hence "AMD64"); Intel later adopted it too (originally called EM64T, now "Intel 64")
- Has more general-purpose registers than 32-bit (16 vs. 8), which helps performance
- Backward compatible: a 64-bit OS/CPU can usually run 32-bit programs, but not the other way around

Q. What is ARM?

ARM stands for **Advanced RISC Machine** (originally *Acorn RISC Machine*, since it was created by Acorn Computers in the UK back in 1985). It's a different family of CPU architecture from x86 , instead of a company/chip-number lineage, ARM is built around a specific design philosophy called **RISC**.

RISC vs. CISC

- x86 is **CISC** (Complex Instruction Set Computing): fewer, more powerful instructions that can each do a lot (like "load from memory, add, and store" in one instruction).
- ARM is **RISC** (Reduced Instruction Set Computing): simpler instructions that each do one small thing, but execute very fast and efficiently.

Key traits of ARM

- **Power-efficient**: this is its biggest claim to fame, which is why it dominates phones, tablets, and battery-powered devices

- **Licensed, not manufactured by one company** ARM Holdings designs the architecture and licenses it out; companies like Apple, Qualcomm, Samsung, and MediaTek build their own chips based on it (e.g., Apple's A-series and M-series chips)
- Also comes in 32-bit and 64-bit variants, just like x86:
    - **AArch32** the older 32-bit ARM instruction set
    - **AArch64** the modern 64-bit ARM instruction set (what almost everything uses today)

Example

- Virtually all smartphones (iPhone, Android)
- Apple's Mac computers since 2020 (M1, M2, M3, M4 chips)
- Raspberry Pi
- Many servers now (AWS Graviton, for example)
- Embedded systems, IoT devices, smartwatches

Q. Difference between Android vs iOS Architecture?

Android Architecture (built on Linux)

Android is structured in layers, from bottom to top:

- **Linux Kernel** the foundation, handling drivers, memory management, process management, and power management.
- **Hardware Abstraction Layer (HAL)** sits above the kernel and provides standard interfaces so the higher layers don't need to know the specifics of the underlying hardware.
- **Android Runtime (ART) and native libraries** this is where your Kotlin/Java code actually runs, compiled to native machine code. Native C/C++ libraries (like SQLite, OpenGL) also live here.
- **Application Framework** provides the building blocks apps use, like the Activity Manager, Window Manager, and Content Providers.
- **Applications** the actual apps (system apps and user-installed apps) sit at the top.

iOS Architecture (built on Darwin/XNU)

iOS also uses a layered approach, from bottom to top:

- **Darwin (XNU Kernel)** a hybrid kernel combining the Mach microkernel with BSD components. Handles low-level tasks like memory, process scheduling, and security.
- **Core OS layer** includes low-level features like security frameworks and the Accelerate framework for hardware-optimized math.

- **Core Services layer** provides fundamental services like Core Data (data persistence), Core Location, and networking.
- **Media layer** handles graphics, audio, and video frameworks (Core Graphics, Core Animation, AVFoundation).
- **Cocoa Touch layer** the top layer, containing UIKit and SwiftUI, which apps use to build their interfaces.

Key Differences

Android's kernel is Linux-based and modified for mobile use, while iOS runs on Darwin/XNU, a hybrid kernel combining Mach and BSD. Android is open source through the Android Open Source Project (AOSP), whereas iOS is closed source and entirely controlled by Apple.

On the language side, Android apps are primarily written in Kotlin or Java, with C/C++ available through the NDK for performance-critical code. iOS apps are written in Swift or Objective-C.

For runtime behavior, Android uses ART (Android Runtime), which compiles app code to native instructions on the device. iOS apps are compiled ahead-of-time directly to native ARM code before they ever reach the device.

App distribution also differs significantly: Android allows installation through Google Play, other third-party app stores, or direct sideloading. iOS has historically restricted installation almost entirely to the App Store, though sideloading has recently become possible in limited regions like the EU due to regulatory pressure.

Hardware-wise, Android runs across many manufacturers (Samsung, Google, Xiaomi, and others) on a wide variety of ARM chips. iOS runs exclusively on Apple's own hardware, using Apple-designed A-series and M-series ARM chips.

Q. Why is C/C++ often used in Game Development?

C/C++ dominates game development for a mix of performance, control, and ecosystem reasons that go back decades and still hold up today.

Performance and speed: Games need to render dozens to hundreds of frames per second while simultaneously handling physics, AI, audio, networking, and input all within a tiny time budget (often under 16 milliseconds per frame for 60 FPS). C/C++ compiles directly to native machine code with minimal overhead, making it one of the fastest languages available. There's no interpreter and no virtual machine sitting between your code and the hardware.

Manual memory management: C/C++ gives developers direct control over memory allocation and deallocation. This matters enormously in games because languages with automatic garbage collection (like Java, C#, or Python) can introduce unpredictable pauses when the garbage collector runs and a sudden freeze mid-gameplay is unacceptable. With C/C++,

developers can pre-allocate memory pools, control exactly when memory is freed, and avoid these stutters entirely.

Low-level hardware access: Games need to talk directly to the CPU and GPU as efficiently as possible. C/C++ allows fine-grained control over things like CPU cache usage, pointer arithmetic, and memory layout (structuring data to be "cache-friendly" for faster access). This level of control simply isn't available in higher-level languages.

Direct access to graphics APIs: Major graphics APIs like DirectX, OpenGL, and Vulkan are written in C/C++ or expose C-style interfaces. Using C/C++ means no translation layer or binding overhead between your game code and the graphics driver, which matters when you're pushing millions of polygons per second.

Game engines are built in it: The industry's dominant engines Unreal Engine, and historically many custom in-house engines at studios like id Software, Naughty Dog, and CD Projekt Red are written in C++. Even engines that let you script in other languages (like C# in Unity) have their core engine written in C/C++ under the hood for performance-critical systems like rendering and physics.

Predictable, deterministic performance: C/C++ doesn't have hidden runtime costs like automatic boxing/unboxing, reflection overhead, or JIT compilation delays. What you write is close to what actually executes, which makes performance easier to reason about and optimize.

Cross-platform portability: Since C/C++ compilers exist for virtually every platform (PC, consoles, mobile), game code written in C/C++ can be ported across platforms with comparatively less friction, especially for performance-critical engine code.

Legacy and tooling: Decades of tooling, profilers, debuggers, and middleware (physics engines like Havok, audio middleware like Wwise) are built with C/C++ compatibility in mind. There's also a massive body of accumulated knowledge and battle-tested code that studios reuse across projects.

What is JVM Architecture / Runtime?

The **JVM (Java Virtual Machine)** is the runtime engine that executes Java bytecode. It's what makes Java's "Write Once, Run Anywhere" promise possible when you compile Java code, it

doesn't compile directly to machine code, it compiles to an intermediate form called **bytecode**, which the JVM then interprets or compiles for the specific machine it's running on. This is also why languages like Kotlin, Scala, and Groovy can run on the JVM too; they all compile down to the same bytecode format.

The JVM architecture has three main subsystems: the **Class Loader**, the **Runtime Data Areas**, and the **Execution Engine**.

1. Class Loader Subsystem

This is responsible for loading `.class` files (compiled bytecode) into memory. It works in three steps:

Loading reads the `.class` file and brings it into memory. There are different loaders involved here: the Bootstrap Class Loader (loads core Java classes like those in `java.lang`), the Extension Class Loader (loads classes from extension directories), and the Application Class Loader (loads classes from your application's classpath).

Linking this has three sub-phases: **Verification** (checks the bytecode is valid and doesn't violate security constraints), **Preparation** (allocates memory for class variables and sets default values), and **Resolution** (replaces symbolic references with direct references).

Initialization actually executes the static initializers and assigns real values to static variables.

2. Runtime Data Areas

This is where the JVM stores data while a program runs. It's divided into several regions:

Method Area stores class-level data: metadata about classes, method information, static variables, and the constant pool. This is shared across all threads.

Heap stores all objects and their instance variables. This is also shared across threads and is the main area the Garbage Collector works on to reclaim memory from objects that are no longer referenced.

Stack each thread gets its own JVM stack, created when the thread starts. It stores frames for method calls local variables, partial results, and data related to method invocation and return. When a method is called, a new frame is pushed; when it returns, the frame is popped.

PC (Program Counter) Register each thread has its own PC register, which keeps track of the address of the currently executing instruction.

Native Method Stack used for executing native (non-Java) code, typically code written in C/C++ and accessed through JNI (Java Native Interface).

3. Execution Engine

This is the part that actually runs the bytecode. It includes:

Interpreter reads and executes bytecode line by line. This is fast to start but slower during execution since it re-interprets the same code every time it runs.

JIT (Just-In-Time) Compiler to fix the interpreter's repeated-execution slowness, the JIT compiler identifies "hot" code (methods called frequently) and compiles them into native machine code on the fly, caching the result. Future calls to that method run at native speed instead of being re-interpreted. This is why Java programs often speed up the longer they run.

Garbage Collector (GC) automatically identifies and removes objects in the heap that are no longer reachable/referenced, freeing up memory without requiring the developer to manually deallocate it. There are different GC algorithms (Serial, Parallel, G1, ZGC, etc.), and the JVM lets you choose or tune which one to use depending on the workload.

Supporting Components

Java Native Interface (JNI) acts as a bridge that allows Java code to call native code written in C/C++, and vice versa. This is how Java can interact with platform-specific features or reuse existing native libraries.

Native Method Libraries the actual native (C/C++) libraries required for native method execution, accessed through JNI.

What is Bytecode?

Bytecode is an intermediate form of code that sits between human-readable source code and machine code that a CPU can execute directly. It's not quite "source code" and it's not quite "native machine code" it's a middle ground designed to be platform-independent while still being fast to execute.

How it fits into the process

When you write Java code, it goes through these stages:

1. **Source code** (`.java` file) the human-readable code you actually write.
2. **Compilation** the Java compiler (`javac`) compiles this source code, but instead of turning it directly into machine code for a specific CPU (like x86 or ARM), it compiles it into **bytecode**, stored in `.class` files.
3. **Bytecode** a set of instructions in a compact, standardized format that isn't tied to any particular hardware or operating system.

4. **Execution** the JVM reads this bytecode and either interprets it line-by-line or uses the JIT compiler to convert it into actual native machine code for the CPU it's running on.

What bytecode actually looks like

Bytecode consists of instructions called **opcodes** each one is a single byte (hence the name "bytecode") representing an operation, sometimes followed by operands (extra data the operation needs). For example, there are opcodes for things like pushing a value onto the stack, adding two numbers, calling a method, or returning from a method.

If you compile a simple Java file and inspect the `.class` file with a tool like `javap -c`, you'd see output like:

0: iconst_5

1: istore_1

2: iconst_3

3: istore_2

4: iload_1

5: iload_2

6: iadd

7: istore_3

This is bytecode for `int a = 5; int b = 3; int c = a + b;` each line is a low-level instruction operating on a virtual stack machine.

Why bytecode exists (the key benefit)

Platform independence. A CPU can only execute instructions in its own native machine code (x86 machine code is different from ARM machine code). If Java compiled directly to machine code, you'd need a different compiled version of your program for every CPU architecture and OS exactly like C/C++ does.

Bytecode solves this by adding a layer of abstraction:

- The **compiler** only needs to know how to turn Java into bytecode once.
- The **JVM** (which is different for each platform Windows, Linux, macOS, ARM, x86) knows how to turn that same bytecode into native instructions for its specific machine.

So the same `.class` file can run unmodified on any device that has a JVM installed, which is the literal meaning of "Write Once, Run Anywhere."

What is Time Complexity and Space Complexity in Java?

Time Complexity and **Space Complexity** are ways to measure how efficient an algorithm is not in exact seconds or bytes, but in terms of how its resource usage *grows* as the input size grows. This is universal across all languages, but let's ground it with Java examples since that's the context you've been building.

Time Complexity

Time complexity measures how the **runtime** of an algorithm grows relative to the size of its input (usually called `n`). It's not about literal seconds (that depends on your CPU, JVM warm-up, JIT compilation, etc.) it's about the number of operations relative to input size, expressed using **Big O notation**.

Common Time Complexities (from best to worst)

Notation	Name	Example
O(1)	Constant	Accessing an array element by index
O(log n)	Logarithmic	Binary search
O(n)	Linear	Looping through an array once
O(n log n)	Linearithmic	Efficient sorting (Merge Sort, Quick Sort avg case)
$O(n^2)$	Quadratic	Nested loops (Bubble Sort, Selection Sort)
$O(n^3)$	Cubic	Triple nested loops

$O(2^n)$	Exponential	Recursive Fibonacci without memoization
O(n!)	Factorial	Generating all permutations

Space Complexity

Space complexity measures how much **memory** an algorithm needs relative to input size this includes both:

Auxiliary space extra space used by the algorithm itself (temporary variables, data structures created during execution)

Input space space taken by the input data itself (usually not counted, since it exists regardless of the algorithm)

What is Spring Boot?

Spring Boot is a framework built on top of the **Spring Framework** that makes it dramatically easier to build Java web applications, REST APIs, and microservices. It was created by Pivotal (now part of VMware) specifically to eliminate the tedious setup and configuration that traditional Spring applications required.

The Problem It Solves

Before Spring Boot, setting up a Spring application meant writing a lot of manual configuration XML files or Java config classes to wire up beans, set up a database connection, configure a web server, and so on. Just getting a basic "Hello World" web app running could take dozens of lines of boilerplate configuration before you wrote a single line of actual business logic.

Spring Boot's philosophy is **"convention over configuration"** it makes sensible assumptions about what you probably want, configures it automatically, and only asks you to override things when your needs differ from the defaults.

Features:

1. Auto-Configuration Spring Boot scans what's on your classpath (which libraries you've added) and automatically configures your application based on that. If it sees a database driver,

it assumes you want a database connection and configures one. If it sees Spring MVC, it sets up a web server. You write far less setup code because Boot infers it from context.

2. Embedded Servers Traditional Java web apps needed to be packaged as a `.war` file and deployed onto an external server like Tomcat. Spring Boot embeds a server (Tomcat, Jetty, or Undertow) directly inside your application. You just run your app as a standalone `.jar` file `java -jar myapp.jar` and it starts its own web server internally. No separate server installation needed.

3. Starter Dependencies Instead of manually figuring out which of 15 different libraries you need for, say, building a web app with a database, Spring Boot provides "starter" packages that bundle everything together. For example:

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-web</artifactId>

</dependency>

This one dependency pulls in everything needed for a web application embedded Tomcat, Spring MVC, JSON handling (Jackson), and more all with compatible versions already figured out for you.

4. Production-Ready Features (Actuator) Spring Boot Actuator gives you built-in endpoints for monitoring and managing your app in production health checks, metrics, environment info without writing any of that yourself.

5. Minimal Boilerplate A complete, runnable Spring Boot web application can be as small as this:

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

import org.springframework.web.bind.annotation.GetMapping;

import org.springframework.web.bind.annotation.RestController;

@SpringBootApplication

@RestController

```
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }

    @GetMapping("/hello")
    public String hello() {
        return "Hello, World!";
    }
}
```

Run this, and you have a working web server listening on port 8080 that responds to `/hello` with no XML config, no external server setup, no manual dependency wiring. The `@SpringBootApplication` annotation alone triggers auto-configuration, component scanning, and enables the app to be run standalone.

Spring Boot vs. Traditional Spring

	Traditional Spring	Spring Boot
Configuration	Manual (XML or extensive Java config)	Auto-configured based on classpath
Server	External (deploy `.war` to Tomcat)	Embedded (`.jar` runs standalone)
Dependency management	Manually match compatible versions	Starter dependencies handle this

Setup time	Hours for basic project	Minutes
Monitoring	Build it yourself	Actuator provides it out of the box
Philosophy	Flexible, but verbose	Opinionated defaults, but still customizable